

Zoho Books Bank-Statement Automation

Complete Deployment Guide

GCP Cloud Run Backend · Zoho Books Widget Extension · OneDrive / Microsoft Graph Integration

Project: Zoho-Agent-Cybeknight
Prepared for: Ravindra Nallan
Date: September 28, 2026

CONFIDENTIAL — contains configuration and infrastructure details only.
All credential values in this document are PLACEHOLDERS. Never store real secrets in this file.

1. Overview & Architecture

This system automates reading bank statements stored on OneDrive, using AI to categorize each transaction, and posting the categorized entries into Zoho Books as journals, expenses, or bank transactions. It is operated entirely from inside Zoho Books through a set of custom widgets, which call a backend service running on Google Cloud Run.

1.1 Components

- GCP Cloud Run backend (Flask app, `gcp_categorize_service/main.py`) — the single service that talks to OneDrive, Zoho Books, and the Gemini AI model. All widgets call this backend over HTTPS.
- Zoho Books widget extension — four widgets embedded inside Zoho Books: Categorize, Post Entries, Reconcile, and Supporting Docs Attach.
- Microsoft OneDrive / SharePoint — where raw bank statement files are stored, organized by entity/organization in dedicated folders.
- Zoho Books API — the system of record; the backend reads chart of accounts, bank accounts, and custom modules, and writes journals/expenses/bank transactions.
- Google Gemini API — used optionally for an AI-assisted reference/description preview feature during categorization.

1.2 High-level data flow

1. A bank statement file is uploaded to a per-organization folder on OneDrive.
2. The user opens the Categorize widget inside Zoho Books, selects an organization, and triggers a run.
3. The Cloud Run backend downloads the statement from OneDrive, parses transactions, and classifies each one (using categorization rules stored in a Zoho custom module, optionally assisted by Gemini).
4. Categorized rows are written back to the statement workbook and re-uploaded to OneDrive as a checkpoint.
5. The user opens the Post Entries widget, reviews the categorized transactions, and posts them — the backend creates the corresponding journals / expenses / bank transactions in Zoho Books via the Zoho Books API.
6. The Reconcile and Supporting Docs Attach widgets provide follow-up workflows: matching bank transactions and attaching supporting documentation to posted entries.

1.3 Why a Cloud Run backend (not just widget JavaScript)

Zoho widgets run inside a sandboxed iframe with limited execution time and no server-side credential storage. The heavy lifting — talking to Microsoft Graph, running the Gemini model, manipulating Excel workbooks with LibreOffice, and holding long-lived OAuth refresh tokens — is done by the Cloud Run service instead. The widgets are thin clients that call this backend with a shared API key.

2. Prerequisites

Before starting a fresh deployment (or redeploying after a GCP project migration/teardown), make sure you have access to all of the following:

- A Google Cloud Platform project with billing enabled, and the `gcloud` CLI installed and authenticated (`gcloud auth login`).

- Owner or Editor access to the target Zoho Books organization(s), plus access to the Zoho API Console (api-console.zoho.com) under the same Zoho account.
- A Microsoft 365 / Azure AD tenant where you can register an application and grant admin consent for application-level Graph permissions (this typically requires a Global Administrator or Application Administrator role).
- Access to the OneDrive/SharePoint location where bank statements are stored, to confirm folder paths and (if needed) resolve the Drive ID.
- Optional: a Google AI Studio / Gemini API key, if the AI reference-preview feature is desired.
- Access to edit the Zoho Books widget extension in the Zoho Developer Console (to update endpoint URLs and the allowed-domains / Content-Security-Policy list).

This guide uses placeholder values everywhere a real credential would go (e.g. <your-client-id>, <your-api-key>). Replace these with real values only inside GCP Secret Manager or the Zoho/Azure consoles — never paste real secrets into chat, source control, or this document.

3. Part A — GCP Cloud Run Backend

This is the core service. It is a Flask application (main.py) packaged with a custom Dockerfile (needed because it installs LibreOffice for headless Excel recalculation) and deployed to Cloud Run.

3.1 Enable required GCP APIs

```
gcloud config set project <your-gcp-project-id>
```

```
gcloud services enable \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  secretmanager.googleapis.com \
  cloudbuild.googleapis.com \
  storage.googleapis.com
```

3.2 Zoho Books OAuth (Self Client) setup

The backend authenticates to Zoho Books using a Self Client refresh token — a long-lived credential generated once and stored in Secret Manager. Using the correct OAuth scope is critical: a refresh token generated with a narrower scope (e.g. only ZohoBooks.settings.READ) will cause every other endpoint to fail with an authorization error, even though the token itself is valid.

Known incident: a mismatched/narrow-scoped refresh token produced {"code":57,"message":"You are not authorized to perform this operation"} on /organizations even though the account itself had full access. The fix was regenerating the Self Client token with the single, full scope below.

Required scope

```
ZohoBooks.fullaccess.all
```

This single scope covers everything the backend needs: organizations, contacts, journals, expenses, bank transactions, vendor payments, bank accounts, settings/taxes, chart of accounts, and custom modules.

Steps

1. Go to <https://api-console.zoho.com> and sign in with the Zoho account that owns/administers the target organization(s).

2. Click Add Client → Self Client.
3. Note the generated Client ID and Client Secret — these map to ZOHO_CLIENT_ID and ZOHO_CLIENT_SECRET.
4. Go to the Self Client's "Generate Code" tab. Enter the scope ZohoBooks.fullaccess.all, choose a time duration (e.g. 10 minutes), and generate the grant/authorization code.
5. Immediately exchange the grant code for tokens (it expires within minutes):

```
curl -X POST https://accounts.zoho.com/oauth/v2/token \
-d "grant_type=authorization_code" \
-d "client_id=<your-zoho-client-id>" \
-d "client_secret=<your-zoho-client-secret>" \
-d "redirect_uri=https://www.zoho.com/books" \
-d "code=<the-grant-code-from-step-4>"
```

The JSON response includes a refresh_token — this is the value that maps to ZOHO_REFRESH_TOKEN. It does not expire under normal use (Zoho refresh tokens are long-lived), so this step is only needed once per deployment, or whenever the token is revoked/regenerated.

6. Find the Organization ID: in Zoho Books, go to Settings → Organization Profile, or call GET <https://www.zohoapis.com/books/v3/organizations> with an access token. This maps to ZOHO_ORGANIZATION_ID.

If you also use the Zoho Books MCP connector (or any other integration) against the same account, it uses a completely separate OAuth session — it will keep working even if the Cloud Run service's refresh token is broken, and vice versa. This is useful for isolating whether an authorization error is account-wide or specific to this deployment's token.

3.3 Microsoft Graph (OneDrive) app registration

OneDrive access uses an entirely separate authentication system from Zoho: an Azure AD app registration with application (not delegated) Graph permissions, authenticated via client-credentials flow. There is no user sign-in step and no refresh token — the backend authenticates as the application itself.

Steps

1. In the Azure Portal, go to Azure Active Directory → App registrations → New registration. Give it a name (e.g. "zoho-bank-automation") and register it.
2. Note the Application (client) ID and Directory (tenant) ID — these map to MS_CLIENT_ID and MS_TENANT_ID.
3. Go to Certificates & secrets → New client secret. Copy the secret VALUE immediately (it is only shown once) — this maps to MS_CLIENT_SECRET.
4. Go to API permissions → Add a permission → Microsoft Graph → Application permissions, and add:
 - Files.Read.All (read-only) or Files.ReadWrite.All (required — the backend renames files to mark them .COMPLETED after posting)
 - Sites.Read.All or Sites.ReadWrite.All if the statements live on a SharePoint site rather than a personal/shared OneDrive
5. Click Grant admin consent for <tenant> — application permissions do not take effect until an administrator explicitly consents.
6. Resolve the Drive ID for the statements location. This is the step most prone to error — see the warning box below.

Known incident: MS_DRIVE_ID was set to a garbled value on more than one occasion — once with a human-readable site/drive name accidentally concatenated in front of the real ID (e.g. "my-site-

name/igfR_twTE-..."), and once with a guessed 'b!' prefix pasted onto a value that was still too short to be a genuine SharePoint drive ID. A real SharePoint drive ID returned by Graph is a long, opaque, base64-like string (commonly 100+ characters, frequently starting with 'b!'). If /list-files or any OneDrive call returns 400 BadRequest or 'invalidRequest: the provided drive id appears to be malformed', do not guess or hand-edit the value — re-fetch it authoritatively (see below) and read back the stored secret with gcloud secrets versions access to confirm no extra characters were pasted in.

To resolve the correct Drive ID authoritatively, use Microsoft Graph Explorer (<https://developer.microsoft.com/graph/graph-explorer>) signed in as an account with access to the site:

```
# Find the SharePoint site
GET https://graph.microsoft.com/v1.0/sites?search=<site-name>
```

```
# List drives on that site to get the drive id
GET https://graph.microsoft.com/v1.0/sites/{site-id}/drives
```

Alternatively, resolve it from a OneDrive sharing link by base64url-encoding the link into a 'u!' token and calling /shares/{token}/driveItem, which returns the parentReference.driveId for that item.

7. Confirm the folder path used inside that drive (e.g. "BankStatements/Dubai") — this maps to MS_STATEMENTS_FOLDER_PATH and is per-organization via the org-folder-map custom module (see Part B).

3.4 Google Gemini API key (optional)

The "AI Reference Preview" feature in the Categorize widget uses Google's Gemini model via the google-genai SDK. If GEMINI_API_KEY is not set, this feature is silently disabled — the rest of the categorization pipeline (rule-based classification) continues to work normally, and no error is raised.

1. Go to <https://aistudio.google.com/apikey> and create an API key.
2. Store it as GEMINI_API_KEY (see Secret Manager table below).

3.5 Secret Manager — full reference

Store every credential in GCP Secret Manager rather than as plain environment variables, and reference them from Cloud Run with --set-secrets. The table below lists every configurable value the backend reads from its environment.

Env var / Secret name	Purpose	Required
CATEGORIZE_API_KEY	Shared secret the widgets send as an API key header/param on every backend call	Yes
ZOHO_CLIENT_ID	Self Client ID from api-console.zoho.com	Yes
ZOHO_CLIENT_SECRET	Self Client secret	Yes
ZOHO_REFRESH_TOKEN	Refresh token generated with ZohoBooks.fullaccess.all scope	Yes
ZOHO_ORGANIZATION_ID	Target Zoho Books organization ID	Yes
MS_TENANT_ID	Azure AD tenant (directory) ID	Yes
MS_CLIENT_ID	Azure AD app registration client ID	Yes
MS_CLIENT_SECRET	Azure AD app registration client secret	Yes

Env var / Secret name	Purpose	Required
MS_DRIVE_ID	Resolved SharePoint/OneDrive drive ID (see 3.3)	Yes
GEMINI_API_KEY	Google Gemini API key for AI reference preview	No — feature disables silently if absent
MS_STATEMENTS_FOLDER_PATH	Base folder path for statements (default: BankStatements/Dubai)	No, has default
ENTITIES_MODULE_API_NAME	Custom module API name for entities (default: cm_entities)	No, has default — legacy
ORG_FOLDER_MODULE_API_NAME	Custom module mapping org → OneDrive folder (default: cm_orgidonedrivemap)	No, has default
ORG_FOLDER_ORGID_FIELD	Field name for org ID in that module (default: cf_orgidonedrivemap)	No, has default
ORG_FOLDER_PATH_FIELD	Field name for folder path in that module (default: cf_one_drive_folder)	No, has default
BANK_ACCOUNT_TYPE_VALUES	Comma-separated bank account type filter (default: bank)	No, has default
CATEGORIZATION_ENTITY_FIELD	Custom field used to tag entity on categorization records (default: cf_entity)	No, has default
BANK_CHARGES_VAT_TAX_RATE_NAME	Tax rate name applied to bank-charge VAT postings	No
BANK_CHARGES_VAT_TAX_RATE_PERCENTAGE	Tax rate percentage applied to bank-charge VAT postings	No
REFERENCE_AI_PROMPT_MODULE_API_NAME	Custom module holding the AI reference prompt (default: cm_referenceaiprompt)	No, has default
REFERENCE_AI_PROMPT_FIELD	Field holding the prompt text (default: cf_ai_prompt)	No, has default
RELATED_ENTITIES_JSON	JSON mapping of related entities used during categorization	No
LOAN_PROVIDERS_JSON	JSON list of known loan provider names for classification	No
PORT	Port the Flask/gunicorn server listens on (Cloud Run sets this automatically)	No, Cloud Run default

anthropic-api-key and trigger-secret are names seen in the historical secret inventory from before the last GCP project migration. The current main.py does not read ANTHROPIC_API_KEY (Gemini replaced it) or a trigger secret (the old Cloud Scheduler-triggered main_gcf.py architecture has been replaced by the widget-driven flow). Treat these two as legacy — safe to omit on a fresh deployment unless you have confirmed a code path still needs them.

Secret naming convention has drifted over the project's history: the original DEPLOY.md (see Appendix C) named secrets identically to their env vars (MS_TENANT_ID, MS_CLIENT_ID, ...), while the secret inventory confirmed during later incident debugging used kebab-case names (ms-tenant-id, zoho-client-id, ...). Either convention works as long as the --set-secrets mapping on the Cloud Run service is

consistent with whatever the secrets are actually named — run `gcloud secrets list` to see what currently exists in your project before assuming either style.

Create every secret the backend needs (values are placeholders — substitute your real values, and never paste real secret values into chat, source control, or this document):

```
export TRIGGER_SECRET=$(openssl rand -hex 24)
```

```
echo "Save this -- you'll paste it into the Zoho button script in Step 6: ${TRIGGER_SECRET}"
```

```
echo -n "<Your TRIGGER_SECRET>" | gcloud secrets create categorize-api-key --data-file=-
echo -n "<your-zoho-client-id>" | gcloud secrets create zoho-client-id --data-file=-
echo -n "<your-zoho-client-secret>" | gcloud secrets create zoho-client-secret --data-file=-
echo -n "<your-zoho-refresh-token>" | gcloud secrets create zoho-refresh-token --data-file=-
echo -n "<your-zoho-organization-id>" | gcloud secrets create zoho-organization-id --data-file=-
echo -n "<your-ms-tenant-id>" | gcloud secrets create ms-tenant-id --data-file=-
echo -n "<your-ms-client-id>" | gcloud secrets create ms-client-id --data-file=-
echo -n "<your-ms-client-secret>" | gcloud secrets create ms-client-secret --data-file=-
echo -n "<your-ms-drive-id>" | gcloud secrets create ms-drive-id --data-file=-
```

Optional — only needed if the AI reference-preview feature is used:

```
echo -n "<your-gemini-api-key>" | gcloud secrets create gemini-api-key --data-file=-
```

If any of these secrets already exist (e.g. from another pipeline's GCP deployment, or the original DEPLOY.md setup in Appendix C), reuse them instead of creating duplicates: `gcloud secrets versions add <secret-name> --data-file=-` adds a new version to an existing secret rather than failing with 'already exists'.

To rotate a single secret later, add a new version and grant the Cloud Run service account access if this is the secret's first version:

```
echo -n '<new-value>' | gcloud secrets versions add zoho-refresh-token --data-file=-
```

```
gcloud secrets add-iam-policy-binding zoho-refresh-token \
--member="serviceAccount:<PROJECT_NUMBER>-compute@developer.gserviceaccount.com" \
--role="roles/secretmanager.secretAccessor"
```

Cloud Run only re-reads a secret's value when a container instance STARTS. Adding a new secret version does not retroactively update an already-running instance. Either explicitly redeploy (`gcloud run deploy ...` again) or wait for a natural scale-to-zero/cold-start cycle — and confirm the Cloud Run service references the secret as its 'latest' version (not a pinned version number) or the new version will never be picked up automatically. Check with: `gcloud run services describe <service> --format="value(spec.template.spec.containers[0].env)"`

3.6 Dockerfile and unicorn timeout

This service cannot use plain Cloud Run buildpacks because it needs LibreOffice installed for headless Excel formula recalculation (used when writing categorized results back into the statement workbook). It must be built from a custom Dockerfile.

```
FROM python:3.11-slim
```

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    libreoffice-calc \
    && rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

COPY . .

```
# Long timeout: /post-transactions can stream for a large batch of entries.
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "--workers", "1", \
    "--timeout", "1800", "main:app"]
```

There are TWO independent timeout ceilings that must both be raised together: gunicorn's own --timeout in the Dockerfile CMD, and Cloud Run's own --timeout flag passed to gcloud run deploy. Whichever one is lower wins. A mismatch here previously caused long /post-transactions runs to be SIGKILLED mid-stream by gunicorn even though Cloud Run's own timeout was set high enough. Both are set to 1800s (30 minutes) in this deployment.

requirements.txt must include at least: Flask, gunicorn, requests, google-genai==2.17.0, openpyxl (or equivalent), and any Microsoft Graph / Zoho HTTP client dependencies used by the client modules.

3.7 Build and deploy to Cloud Run

```
cd gcp_categorize_service/

gcloud builds submit --tag \
    <region>-docker.pkg.dev/<your-gcp-project-id>/<repo-name>/bank-statement-automation

gcloud run deploy bank-statement-automation \
    --image=<region>-docker.pkg.dev/<your-gcp-project-id>/<repo-name>/bank-statement-automation \
    --region=<region> \
    --platform=managed \
    --allow-unauthenticated \
    --memory=1Gi \
    --timeout=1800 \
    --set-secrets=CATEGORIZE_API_KEY=categorize-api-key:latest,\
    ZOHO_CLIENT_ID=zoho-client-id:latest,\
    ZOHO_CLIENT_SECRET=zoho-client-secret:latest,\
    ZOHO_REFRESH_TOKEN=zoho-refresh-token:latest,\
    ZOHO_ORGANIZATION_ID=zoho-organization-id:latest,\
    MS_TENANT_ID=ms-tenant-id:latest,\
    MS_CLIENT_ID=ms-client-id:latest,\
    MS_CLIENT_SECRET=ms-client-secret:latest,\
    MS_DRIVE_ID=ms-drive-id:latest,\
    GEMINI_API_KEY=gemini-api-key:latest
```

On success, gcloud prints the service URL, e.g. `https://bank-statement-automation-
<hash>.<region>.run.app`. This URL is what every widget's GCP_BASE_URL constant must point to (see Part C).

Every time the project is migrated to a new GCP project (as happened previously), the service URL changes. This means the widget extension's CONFIG constants AND its Content-Security-Policy connect-src allowlist both need to be updated to the new hostname — missing the CSP update is what caused a Safari-only failure after a past migration (Chrome had a stale cached policy that happened to still work; Safari enforced the new one). See section 5.4.

3.8 Verify the backend

Run these checks after every deploy, before touching the widgets:

```
# Health / auth check
curl -s -o /dev/null -w "%{http_code}\n" \
    -H "x-api-key: <your-categorize-api-key>" \
    https://<your-service-url>/organizations
```

```
# Should return the list of Zoho Books organizations, confirming both
# the API key check AND the Zoho refresh token/scope are correct.
```

- A 401/403 here usually means CATEGORIZE_API_KEY mismatch between the secret and what the widget sends.
- A Zoho {"code":57,...} error inside a 200/500 response body means the refresh token scope is wrong — regenerate per section 3.2.
- Check Cloud Run logs for any request: `gcloud run services logs read bank-statement-automation --region=<region> --limit=50`

4. Part B — Zoho Custom Modules

The backend depends on a handful of custom modules inside Zoho Books to store configuration and working data that doesn't belong in standard modules. These must exist (with matching API names) before the backend is used.

Module API name	Purpose	Key fields
cm_categorisation	Stores each categorized transaction row awaiting posting	cf_entity, cf_date, amount, description, category, status
cm_orgidonedrivemap	Maps a Zoho Books organization to its OneDrive folder	cf_orgidonedrivemap (org ID), cf_one_drive_folder (path)
cm_entities	Legacy entity list — superseded by org-folder-map in the current flow; retained for backward compatibility	entity name, code
cm_referenceaiprompt	Stores the prompt text sent to Gemini for the AI reference-preview feature	cf_ai_prompt
cm_bank_transaction_testing	Sandbox module used during development/testing of transaction posting logic	varies

Module and field API names are configurable via environment variables (see the Secret Manager table in 3.5) — if your Zoho org already has similarly-named modules for another purpose, override the *_MODULE_API_NAME / *_FIELD variables rather than renaming existing modules.

Zoho's custom "Date & Time" field type is strict about input format: it rejects ISO-8601 strings that include a 'T' separator or a timezone offset (e.g. 2026-09-28T11:54:05+0530 fails with {"code":6,"message":"Invalid date time ... specified."}). Only a plain 'YYYY-MM-DD HH:MM' or 'YYYY-MM-DD HH:MM:SS' string (no 'T', no offset) is accepted. Any code path that writes to a Date & Time custom field (such as cf_date) must build the string manually from local date/time components rather than calling .toISOString().

5. Part C — Zoho Books Widget Extension

The widget extension is what end users see and interact with inside Zoho Books. It is a set of four widgets, each a self-contained HTML/JS/CSS bundle, that call the Cloud Run backend.

Files	Widget	Purpose
-------	--------	---------

Files	Widget	Purpose
widget.html / widget.js	Categorize	Selects org, lists statement files from OneDrive, runs AI categorization, writes results back to the workbook.
post_widget.html / post_widget.js	Post Entries	Reviews categorized transactions and posts them to Zoho Books as journals/expenses/bank transactions; also supports deleting postings.
reconciliation_widget.html / .js / .css	Reconcile	Matches posted bank transactions against bank statement lines for reconciliation.
supporting_docs_widget.html / .js / .css	Supporting Docs Attach	Attaches supporting documentation files to posted entries.

5.1 CONFIG constants

Each widget file defines its backend endpoint URLs and shared API key near the top of the .js file. After every backend redeploy (new service URL) or API key rotation, these must be updated in every widget file and the extension republished. Example (values are placeholders):

```
const GCP_BASE_URL = "https://<your-service-url>.<region>.run.app";
const GCP_ORGANIZATIONS_ENDPOINT = GCP_BASE_URL + "/organizations";
const GCP_ORG_FOLDER_MAP_ENDPOINT = GCP_BASE_URL + "/org-folder-map";
const GCP_LIST_FILES_ENDPOINT = GCP_BASE_URL + "/list-files";
const GCP_BANK_ACCOUNTS_ENDPOINT = GCP_BASE_URL + "/bank-accounts";
const GCP_CATEGORIZATION_RULES_ENDPOINT = GCP_BASE_URL + "/categorization-rules";
const GCP_POST_TRANSACTIONS_ENDPOINT = GCP_BASE_URL + "/post-transactions";
const GCP_POST_BANK_CHARGES_ENDPOINT = GCP_BASE_URL + "/post-bank-charges";
const GCP_DELETE_POSTINGS_ENDPOINT = GCP_BASE_URL + "/delete-postings";
const GCP_API_KEY = "<your-categorize-api-key>"; // must match CATEGORIZE_API_KEY secret
```

Never commit or paste the real GCP_API_KEY value into documentation, chat, or version control. Treat it exactly like any other secret — rotate it via Secret Manager and update all four widget files together if it is ever exposed.

5.2 Zoho custom module reference in the widgets

The Categorize widget also references the categorization module directly for record lookups: `const CATEGORIZATION_MODULE_API_NAME = "cm_categorisation";` — keep this in sync with the module's actual API name in Zoho Books.

5.3 Packaging and uploading the extension

1. In the Zoho Developer Console (or via the zet CLI if the extension was scaffolded that way), open the extension project containing these widget files.
2. Update all CONFIG constants as above.
3. Validate/package the extension (zet pack, or the console's built-in validate step).
4. Upload the new package as a new version and publish/install it into the target Zoho Books organization(s).

If the extension was scaffolded with Zoho's zet CLI, the everyday local-dev/republish loop is:

```
zet run # serve the extension locally for live testing inside Zoho Books
zet pack # package it into a .zip for upload as a new version
```

5.4 Content-Security-Policy — connect-src allowlist

This is a real incident, not a hypothetical: after migrating the Cloud Run service to a new GCP project, the widgets started failing ONLY in Safari with a browser console error: "Refused to connect to https://<new-service-url> because it does not appear in the connect-src directive of the Content Security Policy." Chrome appeared to work at first only because it had cached the old policy/response; Safari enforced the current one immediately. This is a CSP restriction declared by the widget/extension configuration itself — separate from, and enforced BEFORE, any CORS headers the backend sends.

1. If the extension was scaffolded with the zet CLI, open plugin-manifest.json in the extension project and find the cspDomains array (in the Zoho Developer/Extension console UI, this is the equivalent "allowed domains" list under the widget's connection settings).
2. Add the new Cloud Run service origin — the scheme + host only, no trailing path, e.g. https://<your-service-url>.<region>.run.app — to cspDomains / the allowed-domains list.
3. Remove any stale hostnames from a previous GCP project, if desired, once the migration is confirmed complete.
4. Repackage and republish the extension, and hard-refresh (or fully quit/reopen) the browser — do not rely on the widget iframe simply reloading, since some browsers cache the CSP header aggressively.

Test in more than one browser after any backend URL change. A CSP allowlist mismatch can silently work in one browser (cached policy) while failing in another (freshly enforced policy), which is exactly what happened here.

6. Part D — End-to-End Verification Checklist

After a fresh deployment or any redeploy, walk through this checklist in order:

1. Backend health: curl /organizations with the shared API key returns 200 and a real organization list (section 3.8).
2. Open the Categorize widget in Zoho Books (in at least two browsers, including Safari if your users use it). Confirm the organization dropdown populates without a CSP or CORS error in the browser console.
3. Select an organization and confirm the OneDrive folder/file list loads (validates MS_DRIVE_ID, MS_STATEMENTS_FOLDER_PATH, and the org-folder-map module).
4. Run categorization on a small test statement and confirm rows are classified and the workbook is written back to OneDrive.
5. Open the Post Entries widget, confirm categorized rows are visible, and post a small test batch — confirm the journal/expense/bank transaction appears in Zoho Books with the expected date, amount, and account.
6. If posting touches any custom Date & Time field (e.g. cf_date), confirm no {"code":6, Invalid date time...} error appears (section 4).
7. Exercise the Reconcile and Supporting Docs Attach widgets against the same test data.
8. Check Cloud Run logs for unexpected tracebacks during the whole run: gcloud run services logs read bank-statement-automation --region=<region> --limit=100.

Appendix A — Troubleshooting Reference

Drawn from real incidents encountered operating this system. Check here first before re-diagnosing from scratch.

Symptom	Root cause	Fix
Zoho { <code>"code":57,"message":"You are not authorized..."</code> } on any Zoho Books call	Refresh token was generated with a narrower/mismatched OAuth scope	Regenerate the Self Client refresh token with scope <code>ZohoBooks.fullaccess.all</code> (section 3.2) and update the secret + redeploy.
A backend route returns a bare 502 with no useful detail	Route lacked exception logging, so the real traceback never reached Cloud Run logs	Ensure every route wraps its logic in try/except and logs the full traceback (<code>import traceback; print(traceback.format_exc())</code>) before returning the error response.
OneDrive/Graph call returns 400 BadRequest or 'invalidRequest: malformed drive id'	MS_DRIVE_ID secret contains extra text (a site name) or an incomplete/guessed ID	Re-resolve the Drive ID via Graph Explorer or the sharing-link method (section 3.3); verify the stored secret with <code>gcloud secrets versions access <n> --secret=ms-drive-id</code> before redeploying.
Updated a secret but the backend still behaves like the old value	Cloud Run only re-reads secrets at container startup	Explicitly redeploy, or confirm the service references the secret as 'latest' and wait for a scale-to-zero/cold-start cycle (section 3.5).
Widget fails to load organizations in Safari only, browser console shows a connect-src CSP refusal	The widget/extension's own CSP allowlist was not updated after the backend's service URL changed	Add the new Cloud Run hostname to the extension's connect-src / allowed-domains configuration and republish (section 5.4).
Posting fails with <code>{"code":6,"message":"Invalid date time ... specified."}</code>	A custom Date & Time field was sent an ISO string with a 'T' separator or timezone offset	Build the date string manually as plain 'YYYY-MM-DD HH:MM[:SS]' (no 'T', no offset) before writing to the custom field.
A long /post-transactions run is killed mid-stream with no error, entries partially posted	gunicorn's --timeout was lower than Cloud Run's --timeout (or vice versa)	Raise both gunicorn's Dockerfile CMD --timeout and Cloud Run's --timeout flag together (section 3.6); rely on incremental checkpoint-saving in the posting loop as a safety net.
AI reference preview silently does nothing	GEMINI_API_KEY is unset — this is by design, not a bug	Set GEMINI_API_KEY in Secret Manager if the feature is desired (section 3.4); no error will ever surface if it's simply omitted.

Appendix B — GCP Project Migration / Redeploy-from-Scratch Checklist

Use this checklist whenever the backend needs to move to a new GCP project (e.g. after a teardown) or be redeployed end-to-end from nothing.

B.1 GCP setup

- Create/select the new GCP project and enable billing.
- Enable required APIs (section 3.1).
- Create an Artifact Registry repository for the container image.

B.2 Credentials

- Regenerate the Zoho Self Client refresh token with ZohoBooks.fullaccess.all (section 3.2) — do not assume an old token will transfer.
- Confirm the Azure AD app registration and its client secret are still valid (secrets can expire) — rotate if needed (section 3.3).
- Re-resolve MS_DRIVE_ID authoritatively rather than copying an old value that may reference a different site/drive.
- Re-create the Gemini API key if it was tied to the old project's billing (section 3.4).

B.3 Secrets and deploy

- Create all secrets fresh in the new project's Secret Manager (section 3.5) — secrets do not migrate between projects automatically.
- Build and deploy the Cloud Run service (section 3.7); note the NEW service URL.
- Run backend verification (section 3.8).

B.4 Widget extension updates

- Update GCP_BASE_URL and all endpoint constants in all four widget files to the new service URL (section 5.1).
- Update the extension's connect-src / CSP allowlist to the new hostname — this step was missed during at least one past migration and caused a Safari-specific outage (section 5.4).
- Repackage and republish the extension.

B.5 Cleanup of the old project

- Confirm nothing still points at the old project before deleting resources.
- Delete unused Compute Engine persistent disks left behind by stopped VMs — these continue to bill even when the VM is stopped: `gcloud compute disks list`, then `gcloud compute disks delete <disk-name> --zone=<zone>`.
- Delete old Artifact Registry repositories, Cloud Storage buckets, and Cloud Run services once confirmed unused.

B.6 Final verification

- Run the full End-to-End Verification Checklist in Part D, in more than one browser.

Appendix C — Original DEPLOY.md (Historical Reference)

This appendix preserves the project's original `gcp_categorize_service/DEPLOY.md` nearly verbatim. It documents an EARLIER, simpler revision of this same backend — before the Zoho OAuth integration, the Gemini AI step, and the LibreOffice/Dockerfile/gunicorn-timeout requirements existed. At that stage the service only handled Microsoft Graph (OneDrive) access, had a single `/categorize` endpoint called once per bank account, and deployed via Cloud Run's buildpacks (`--source .`) rather than a custom Dockerfile. It is kept here because a few concrete details in it — the exact `plugin-manifest.json` field name, the `zet` CLI commands, and the `list-files-then-categorize` sanity-check pattern — are not restated anywhere else in this guide and remain useful background. Do NOT follow its `--set-secrets` list or `deploy` command as-is for the current service; use Part A instead.

C.1 Original scope note

This is the backend the original "Select Accounts" widget called: once to list what's in the OneDrive statements folder, then once per bank account to actually categorize that account's file. It needed the same Microsoft Graph app-only credentials already used by `onedrive_client.py`, plus one shared secret the widget also needed to know (`CATEGORIZE_API_KEY`).

C.2 One-time setup

```
gcloud config set project YOUR_PROJECT_ID
gcloud services enable run.googleapis.com artifactregistry.googleapis.com secretmanager.googleapis.com
```

C.3 Store secrets (original naming — env-var-identical, not kebab-case)

```
echo -n "YOUR_MS_TENANT_ID" | gcloud secrets create MS_TENANT_ID --data-file=-
echo -n "YOUR_MS_CLIENT_ID" | gcloud secrets create MS_CLIENT_ID --data-file=-
echo -n "YOUR_MS_CLIENT_SECRET" | gcloud secrets create MS_CLIENT_SECRET --data-file=-
echo -n "YOUR_MS_DRIVE_ID" | gcloud secrets create MS_DRIVE_ID --data-file=-
echo -n "pick-a-long-random-string" | gcloud secrets create CATEGORIZE_API_KEY --data-file=-
```

If any of these secrets already existed from another pipeline's GCP deployment, the original guidance was to reuse them with `gcloud secrets versions add SECRET_NAME --data-file=-` instead of `create`.

C.4 Original deploy command (buildpacks, superseded)

```
gcloud run deploy categorize-service \
  --source . \
  --region us-central1 \
  --allow-unauthenticated \
  --set-
secrets="MS_TENANT_ID=MS_TENANT_ID:latest,MS_CLIENT_ID=MS_CLIENT_ID:latest,MS_CLIENT_SECRET=MS_CLIENT_SECRET:latest,MS_DRIVE_ID=MS_DRIVE_ID:latest,CATEGORIZE_API_KEY=CATEGORIZE_API_KEY:latest" \
  --set-env-vars="MS_STATEMENTS_FOLDER_PATH=BankStatements/Dubai"
```

`--allow-unauthenticated` was required so the widget's browser-side `fetch()` could reach the service directly, with the `X-Api-Key` header as the actual access control instead of Cloud Run IAM. This tradeoff still holds in the current deployment (section 3.7 also deploys with `--allow-unauthenticated`, protected the same way by `CATEGORIZE_API_KEY`).

C.5 Wiring the widget to the service (original single-endpoint style)

Two files needed the service URL and API key in this original version:

1. `zoho-books-extension/app/widget.js` — set `GCP_CATEGORIZE_ENDPOINT` to `<Service URL>/categorize` and `GCP_API_KEY` to the same string stored in `CATEGORIZE_API_KEY`.

2. zoho-books-extension/plugin-manifest.json — add the Service URL's origin (scheme + host, no trailing path) to the cspDomains array, or the widget's fetch() call is blocked by Zoho's extension sandboxing before it even leaves the browser.

Then repackage and reload the extension with `zet run / zet pack`.

C.6 Original sanity-check pattern (still a useful template)

First, list what's in the folder (the one-time-per-run call):

```
curl "<Service URL>/list-files" -H "X-API-Key: pick-a-long-random-string"
```

Expected response shape: `{"success": true, "folder_path": "...", "files": ["012001940568_NBF AED_CBK.xlsx", ...]}`. Copy one of the exact file names for the next step.

Then categorize that one file directly by name:

```
curl -X POST "<Service URL>/categorize" \
-H "Content-Type: application/json" \
-H "X-API-Key: pick-a-long-random-string" \
-d '{
  "file_name": "012001940568_NBF AED_CBK.xlsx",
  "rules": [
    {
      "cf_category": "Bank Charges",
      "cf_key_words": "bank charge, service fee",
      "cf_from_account": "Bank Charges Expense",
      "cf_to_account": "NBF AED",
      "cf_type_of_transaction": "Expense"
    }
  ]
}'
```

Expected response: JSON with per-file stats (total, others, by_category), or a clear success: false error if file_name/folder_path didn't exactly match what /list-files returned. The equivalent check against the current service is section 3.8's /organizations check, plus exercising /list-files and /post-transactions the same way with real values.